

Below is a **Jupyter Notebook model + code** you can copy into Jupyter/Google Colab.

It assumes you have:

- March_collection.xlsx = payments file
- ITaxpayer1.xlsx to ITaxpayer9.xlsx = individual taxpayer files
- CTaxpayer.xlsx = corporate taxpayer file
- Default TIN = [9999999999](#)

1. Install and import libraries

```
!pip install pandas numpy scikit-learn tensorflow openpyxl rapidfuzz
import pandas as pd
import numpy as np
import re
from rapidfuzz import fuzz
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
```

2. Load the files

```
collection_file = "March_collection.xlsx"

taxpayer_files = [
    "ITaxpayer1.xlsx",
    "ITaxpayer2.xlsx",
    "ITaxpayer3.xlsx",
    "ITaxpayer4.xlsx",
    "ITaxpayer5.xlsx",
    "ITaxpayer6.xlsx",
    "ITaxpayer7.xlsx",
    "ITaxpayer8.xlsx",
    "ITaxpayer9.xlsx",
    "CTaxpayer.xlsx"
]

collections = pd.read_excel(collection_file)

taxpayer_list = []
for file in taxpayer_files:
    df = pd.read_excel(file)
    taxpayer_list.append(df)

taxpayers = pd.concat(taxpayer_list, ignore_index=True)

print(collections.head())
print(taxpayers.head())
```

3. Clean column names

```
collections.columns = collections.columns.str.strip().str.upper()
taxpayers.columns = taxpayers.columns.str.strip().str.upper()

print(collections.columns)
print(taxpayers.columns)
```

4. Set key column names

Adjust these names if your actual Excel headings are slightly different.

```
COLLECTION_TIN_COL = "KRIN/TIN"
TAXPAYER_TIN_COL = "KRIN/TIN"

COLLECTION_NAME_COL = "PAYER NAME"
TAXPAYER_NAME_COL = "TAXPAYER NAME"

COLLECTION_PHONE_COL = "PHONE"
TAXPAYER_PHONE_COL = "PHONE"

COLLECTION_ADDRESS_COL = "ADDRESS"
TAXPAYER_ADDRESS_COL = "ADDRESS"

DEFAULT_TIN = "9999999999"
```

5. Basic cleaning functions

```
def clean_text(value):
    if pd.isna(value):
        return ""
    value = str(value).upper()
    value = re.sub(r"^[A-Z0-9\s]", " ", value)
    value = re.sub(r"\s+", " ", value).strip()
    return value

def clean_phone(value):
    if pd.isna(value):
        return ""
    value = str(value)
    value = re.sub(r"\D", "", value)
    if value.startswith("234"):
        value = "0" + value[3:]
    return value[-10:] if len(value) >= 10 else value

collections["CLEAN_NAME"] =
collections[COLLECTION_NAME_COL].apply(clean_text)
taxpayers["CLEAN_NAME"] = taxpayers[TAXPAYER_NAME_COL].apply(clean_text)

collections["CLEAN_PHONE"] =
collections[COLLECTION_PHONE_COL].apply(clean_phone)
```

```

taxpayers["CLEAN_PHONE"] = taxpayers[TAXPAYER_PHONE_COL].apply(clean_phone)

collections["CLEAN_ADDRESS"] =
collections[COLLECTION_ADDRESS_COL].apply(clean_text)
taxpayers["CLEAN_ADDRESS"] =
taxpayers[TAXPAYER_ADDRESS_COL].apply(clean_text)

collections[COLLECTION_TIN_COL] =
collections[COLLECTION_TIN_COL].astype(str).str.strip()
taxpayers[TAXPAYER_TIN_COL] =
taxpayers[TAXPAYER_TIN_COL].astype(str).str.strip()

```

6. Select payments with default TIN

```

default_payments = collections[
    collections[COLLECTION_TIN_COL] == DEFAULT_TIN
].copy()

print("Default TIN payments:", len(default_payments))

```

7. Create candidate matches

This creates possible taxpayer matches for each default payment.

```

candidate_rows = []

for payment_index, payment in default_payments.iterrows():
    payer_name = payment["CLEAN_NAME"]
    payer_phone = payment["CLEAN_PHONE"]
    payer_address = payment["CLEAN_ADDRESS"]

    for taxpayer_index, taxpayer in taxpayers.iterrows():
        taxpayer_name = taxpayer["CLEAN_NAME"]
        taxpayer_phone = taxpayer["CLEAN_PHONE"]
        taxpayer_address = taxpayer["CLEAN_ADDRESS"]

        name_score = fuzz.token_sort_ratio(payer_name, taxpayer_name)
        phone_score = 100 if payer_phone != "" and payer_phone ==
taxpayer_phone else 0
        address_score = fuzz.token_sort_ratio(payer_address,
taxpayer_address)

        overall_score = (
            0.50 * name_score +
            0.35 * phone_score +
            0.15 * address_score
        )

        if overall_score >= 50:
            candidate_rows.append({
                "payment_index": payment_index,

```

```

        "taxpayer_index": taxpayer_index,
        "payment_name": payer_name,
        "taxpayer_name": taxpayer_name,
        "name_score": name_score,
        "phone_score": phone_score,
        "address_score": address_score,
        "overall_score": overall_score,
        "probable_krin_tin": taxpayer[TAXPAYER_TIN_COL]
    })

candidates = pd.DataFrame(candidate_rows)

print(candidates.head())
print("Candidate records:", len(candidates))

```

8. Create training labels

For a real business project, you need some manually confirmed matches.
For this notebook, we create a starting rule-based label.

```

def create_label(row):
    if row["phone_score"] == 100 and row["name_score"] >= 70:
        return 1
    elif row["overall_score"] >= 85:
        return 1
    else:
        return 0

candidates["label"] = candidates.apply(create_label, axis=1)

print(candidates["label"].value_counts())

```

9. Prepare data for neural network

```

features = ["name_score", "phone_score", "address_score", "overall_score"]

X = candidates[features].values
y = candidates["label"].values

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)

```

10. Build neural network model

```

model = Sequential([
    Dense(32, activation="relu", input_shape=(X_train.shape[1],)),
    Dropout(0.2),
    Dense(16, activation="relu"),
    Dropout(0.2),
    Dense(8, activation="relu"),
    Dense(1, activation="sigmoid")
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

model.summary()

```

11. Train the model

```

history = model.fit(
    X_train,
    y_train,
    epochs=30,
    batch_size=32,
    validation_split=0.2,
    verbose=1
)

```

12. Evaluate the model

```

y_pred_prob = model.predict(X_test)
y_pred = (y_pred_prob >= 0.5).astype(int)

print(classification_report(y_test, y_pred))

```

13. Predict best KRIN/TIN for each default payment

```

candidate_features = candidates[features].values
candidate_features_scaled = scaler.transform(candidate_features)

candidates["model_confidence"] = model.predict(candidate_features_scaled)

best_matches = (
    candidates.sort_values(
        by=["payment_index", "model_confidence", "overall_score"],
        ascending=[True, False, False]
    )
    .groupby("payment_index")
    .first()
)

```

```

        .reset_index()
    )

best_matches.head()

```

14. Apply prediction back to March collection file

```

collections["PROBABLE_KRIN_TIN"] = ""
collections["MATCH_CONFIDENCE"] = ""
collections["MATCH_STATUS"] = ""

for _, row in best_matches.iterrows():
    idx = row["payment_index"]
    confidence = float(row["model_confidence"])

    collections.loc[idx, "PROBABLE_KRIN_TIN"] = row["probable_krin_tin"]
    collections.loc[idx, "MATCH_CONFIDENCE"] = round(confidence, 4)

    if confidence >= 0.85:
        status = "High confidence match"
    elif confidence >= 0.60:
        status = "Medium confidence - review recommended"
    else:
        status = "Low confidence - manual review required"

    collections.loc[idx, "MATCH_STATUS"] = status

```

15. Export final output

```

output_file = "March_collection_with_probable_KRIN_TIN.xlsx"

collections.to_excel(output_file, index=False)

print("File created:", output_file)

```

16. Optional: Export review-only file

```

review_file = collections[
    collections["MATCH_STATUS"].isin([
        "Medium confidence - review recommended",
        "Low confidence - manual review required"
    ])
]

review_file.to_excel("Manual_review_required.xlsx", index=False)

print("Manual review file created.")

```

Simple explanation for your assignment

This neural network supports payment identity resolution by learning matching patterns between default-TIN transactions and known taxpayer records. It compares payer names, phone numbers, and addresses, then produces a probable TIN with a confidence score. High-confidence matches can be accepted faster, while lower-confidence matches are routed for manual review. This improves reconciliation accuracy, reduces manual workload, supports compliance, and strengthens data integrity in the fintech application.